Modern Cloud Computing

Lecture 18: Modern Resource Management

Chongzhou Fang

Rochester Institute of Technology

Today's Topic

- Cluster management works Published around 2020
  - 5 years after Borg (EuroSys'15)
    - Autopilot: Autoscaling solution at Google (EuroSys'20)
    - Twine: Facebook's cluster management system (OSDI'20)

Part 1: Autopilot

Workload Autoscaling at Google

Rzadca et al., Google — EuroSys 2020

Problem Setup

- Borg asks users for a **task count** and **per-task CPU/RAM limits**
- Limits make performance isolation possible, but estimating them is hard, and the right answer can **drift**:
  - diurnal/weekly cycles, new binaries…
- Exceed memory → task **killed (OOM)**; exceed CPU under contention → **throttled**
- Rational users **over-subscribe**.
  - ~50% average memory utilization in a Google cluster trace
  - Alibaba tasks' peak memory ≤80%

Slack = limit − actual usage.

Autopilot's job:

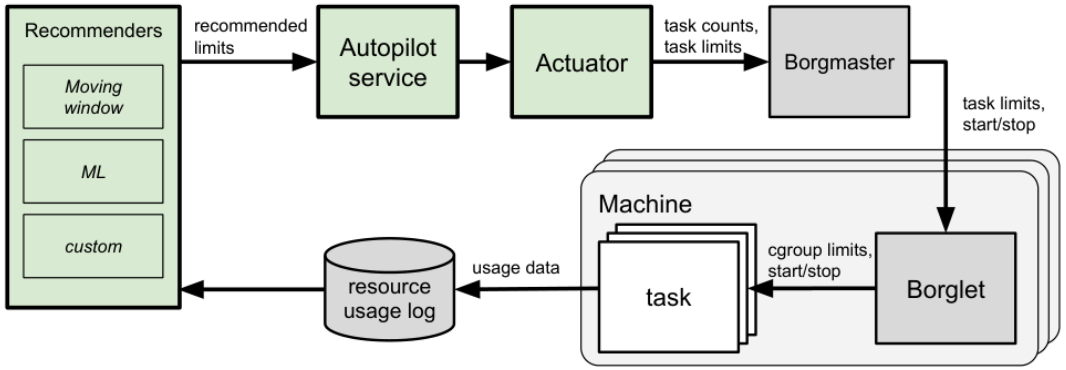
> Shrink slack without causing OOMs or throttling.

Borg Background

- Each cluster (median ≈10K machines) has its own **Borgmaster** (replicated scheduler) and a **Borglet** agent per machine running dozens of tasks
- **CPU limit** in milli-cores: tasks may exceed it while the machine is uncontended; enforced only under contention
- **RAM limit**:
  - **hard** (killed immediately when exceeded)
  - **soft** (may exceed; the Borglet kills over-limit tasks only when the machine is at risk).
- Jobs can change their **task count** (horizontal) and **limits** (vertical) while running, but a task that grows may no longer fit its machine and **evict lower-priority tasks**

Autopilot Architecture

- **Three closed-loop controllers per job**: horizontal (task count), vertical CPU, vertical memory. **No cross-job learning**
- One Autopilot per cluster, built from ordinary Borg jobs:
  - **Recommenders** (3 replicas each) size a job from its usage history
  - **Autopilot service** (elected master) picks a job's recommender and filters its output
  - **Actuator** applies it via the **Borgmaster** → Borglets set cgroup limits, start/stop tasks
- Autopiloted jobs are **>48% of fleet-wide resource usage** at publication time



Input Signal

- Raw per-task series $r_i[\tau]$ at **1 s** resolution (CPU, memory, QPS, ...)
- Monitoring pre-aggregates into **5-minute windows**: a per-task histogram $s_i[t]$ over ≈**400 usage buckets**
  - CPU: counts of samples per bucket (the full distribution)
  - Memory: only the window's **peak** (we must provision for close to peak, because an OOM is worse than throttling)
- Per-job signal: $s[t] = \sum_i s_i[t]$
  - Tasks are treated as interchangeable replicas, so **all tasks get the same limit**

Moving Window Recommender

Then compute one statistic over the decayed histogram:

- **Peak** $S_{\max}$: highest non-empty bucket in the last N samples
- **Weighted average** S_{avg}

$$S_{avg}[t] = \frac{\sum_{\tau=0}^{\infty} w[\tau]\overline{s[t-\tau]}}{\sum_{\tau=0}^{N} w[\tau]},$$

  where

$$w[\tau] = 2^{-\tau/t_{1/2}}$$

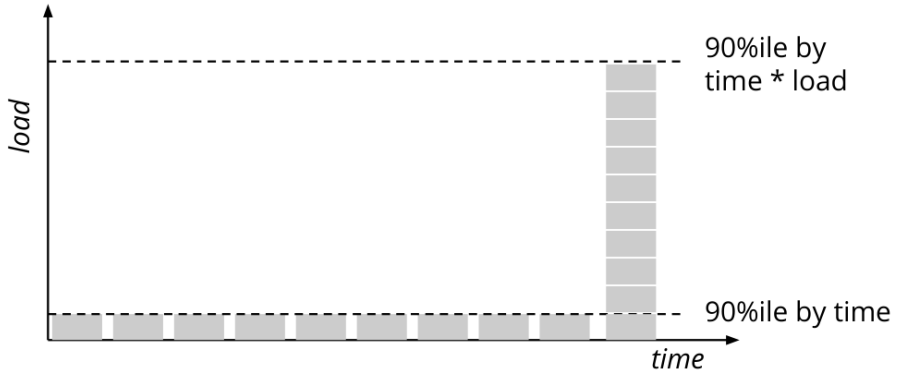
  and $\overline{s[\tau]}$ is the average usage of histogram
- $j-\%$ **ile of adjusted usage** S_{p_j}: weight each sample by its load $b[k]$, then take the j-th percentile

Finally: add a **10–15% safety margin** (less for large limits) and take the **max over the last hour** to reduce fluctuation.

> Goal: increase limits **quickly** on rising usage, decrease them **slowly** after it falls.

Which Statistic for Which Job

- Percentile **by time**: 90% of *samples* fall below the limit
- **Load-adjusted** percentile: 90% of the **signal area** (load × time) is served within the limit
- Example: load 1 for 9 units, then 10 for 1 unit → p90 by time = **1**, load-adjusted p90 = **10**



(Source: Rzadca et al., EuroSys 2020, Fig. 2)

Resource	Job class	Statistic
CPU	batch (tolerates throttling)	S_{avg} (the mean load)
CPU	serving	S_{p95} or S_{p90} of load-adjusted usage, by latency sensitivity
Memory	low OOM tolerance (default for large jobs)	S_{p98}
Memory	minimal OOM tolerance (default for small jobs)	$S_{\max}$
Memory	intermediate	$\max(S_{p60},\ 0.5\,S_{\max})$

Recommender 2: An Ensemble of Simple ML Models

Each **model** m is an arg-min rule with its own **decay rate** d_m and **safety margin** M_m. At each step it tests every candidate limit L (every bucket boundary):

$$o(L)[t] = (1 - d_m)\, o(L)[t-1] + d_m \sum_{j:\, b[j] > L} s[t][j] \qquad u(L)[t] = (1 - d_m)\, u(L)[t-1] + d_m \sum_{j:\, b[j] < L} s[t][j]$$

$$L'_m[t] = \arg\min_L \left(w_o\, o(L)[t] + w_u\, u(L)[t] + w_{\Delta L}\, \Delta(L, L'_m[t-1]) \right), \qquad L_m[t] = L'_m[t] + M_m$$

- **Overrun** cost = lost opportunity (delayed queries); **underrun** cost = infrastructure waste; Δ penalizes **changing the limit** (evictions)
- The recommender tracks each model's smoothed historical cost $c_m[t]$ and picks the cheapest model, with extra penalties for **switching model** or **switching limit**
- Like a **multi-armed bandit**, but with **full observability**: once the next histogram arrives, the cost of *every* arm can be computed (except the rare OOM)

Horizontal Autoscaling: Task Counts

Raw recommendation $n_r[t]$ from one of:

- **CPU utilization**: $n_r[t] = r_S[t]/r^*$ with statistic S (max or p95) over a horizon length T (default 72 h) and an owner-set target utilization r^*
- **Target size**: an owner-supplied function f of monitoring data (e.g., p95 request-handling time, amount of filespace served)

Stabilization policies:

- **Deferred downscaling**: $n_s[t] = \max$ of the last T_d recommendations: upscaling is immediate, downscaling is deferred
- **Slow decay**: remove tasks gradually
- **Defer small changes**; **limit growth** by capping the fraction of still-initializing tasks

Horizontal scaling needs more owner configuration than vertical (zero-config for most jobs) and is used mainly by **large jobs**.

Part 2: Twine

A Unified Cluster Management System for Shared Infrastructure

Tang et al., Facebook — OSDI 2020

Goal: Ubiquitous Shared Infrastructure

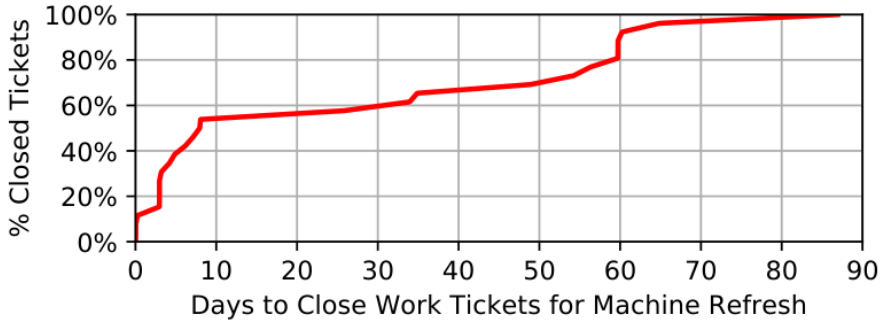
Facebook's fleet used to be **siloed pools of customized machines**, each dedicated to one workload. Twine's goal: one shared pool with **fungible hardware**.

The authors argue existing cluster managers fall short in four ways:

1. They focus on **isolated clusters** → stranded capacity
2. They rarely **consult the application** about lifecycle operations → availability risk
3. They rarely let applications **customize hardware/OS settings** on shared machines → performance loss
4. They prefer **big machines** to stack workloads → under-used big machines waste **power**

Siloed Clusters Strand Capacity

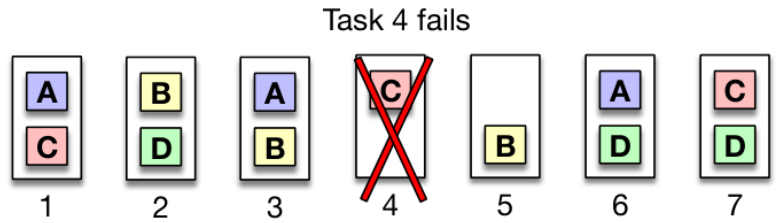
- A cluster ≈ 10K machines with its own isolated control plane; a job is **bound to its cluster**
- Moving jobs needs humans: e.g., power-hungry jobs colocated in one cluster trigger **power capping** until someone relocates them
- Hardware refresh happens 25% of a DC at a time; owners of **thousands of jobs** must migrate out first
- 2016 HW refresh: **P50 = 7.5 days, P100 = 87 days** to close migration tickets: a large part of the cluster sat idle for ~80 days



CDF of time to close job-migration tickets (Source: Tang et al., OSDI 2020, Fig. 1)

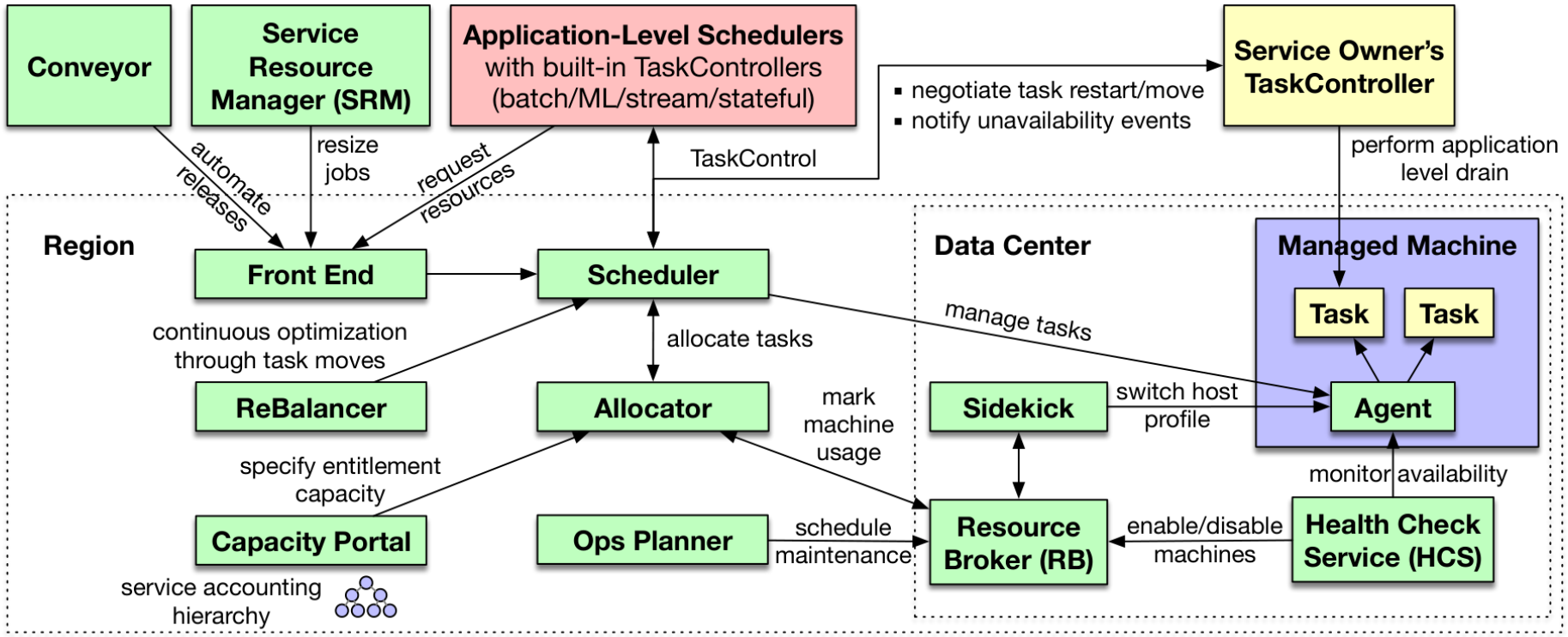
Lifecycle Decisions Without Application Knowledge

- Cluster managers can't see how an application manages its **internal state**
- Shards A–D are replicated across tasks 1–7:
  - Restarting tasks **1 and 3** together for a rollout makes shard **A** lose two replicas → unavailable
  - If task **4** fails, neither task **1** nor **7** may be restarted until shard **C** is rebuilt
- Example: A ZooKeeper ensemble wants to upgrade **followers first, leader last** — otherwise up to *n* leader failovers



(Source: Tang et al., OSDI 2020, Fig. 2)

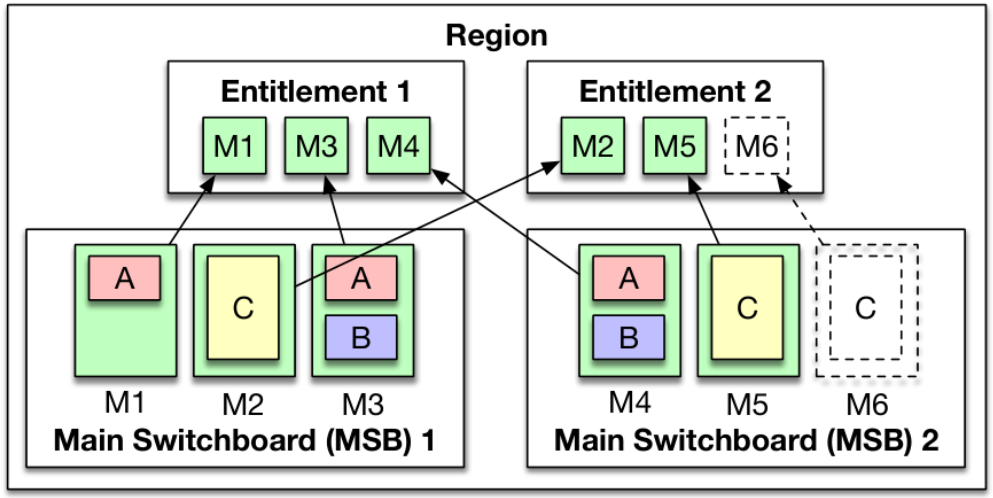
The Twine Ecosystem



(Source: Tang et al., OSDI 2020, Fig. 4) — Terminology: the Twine *scheduler* ≈ Kubernetes *controllers*; the Twine *allocator* ≈ the Kubernetes *scheduler*.

Entitlements

- An **entitlement** = a business unit's quota, as machine counts of a type (e.g., 2,000 Skylake) or **Relative Resource Units** (like AWS ECUs)
- A machine is **free** or assigned to **one** entitlement; it can be **reassigned dynamically**, and an entitlement may span **several DCs**
- A job is bound to an **entitlement** instead of a physical cluster; jobs in an entitlement **stack** on its machines
- Default: spread a job across DCs and **main switchboards (MSBs)**



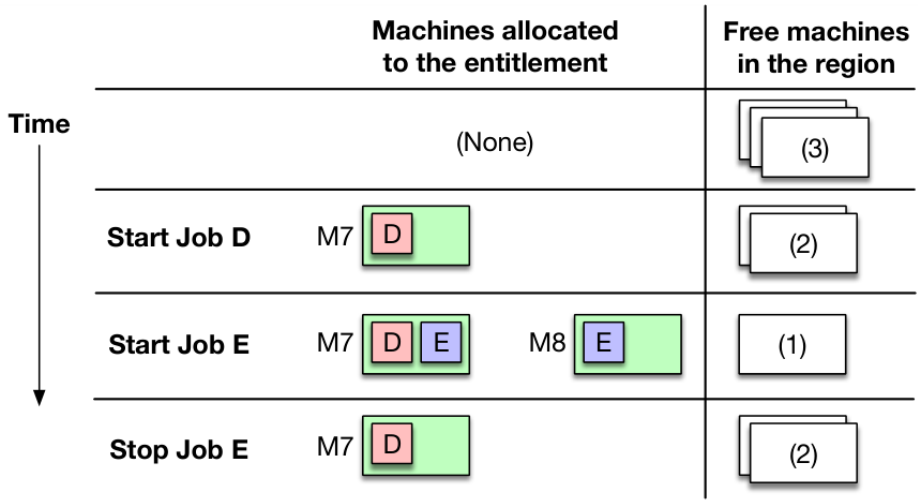
Entitlement 1 = {M1, M3, M4}; jobs A and B stack on M3; as job C grows, M6 is added to Entitlement 2 (Source: Fig. 6)

Allocator and ReBalancer

Allocator: called on job start, resize, or machine failure

- Request = entitlement ID + policy + per-task allocate/free map
- **Hard requirements** ("Skylake, ≥2 RRU CPU, ≥5 GB free") filter an in-memory index of all machines; **soft preferences** (spread across fault domains) sort the rest
- Multi-threaded with **optimistic concurrency control**; per-job result cache with **>99% hit ratio**

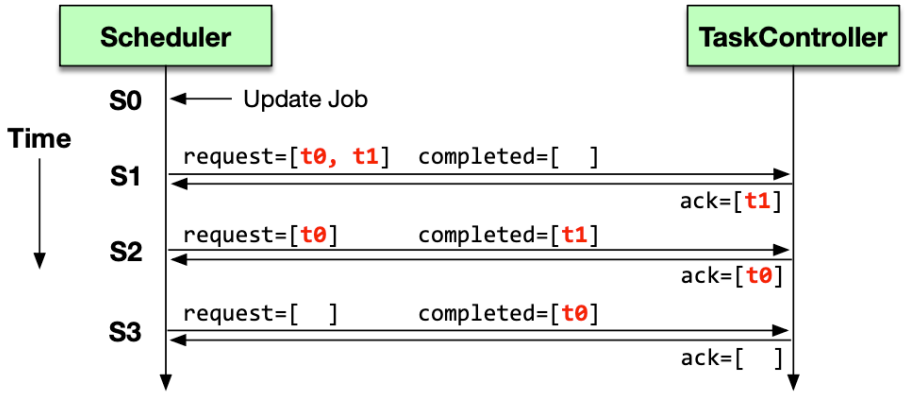
ReBalancer: asynchronous; swaps machines across entitlements and moves tasks to remove CPU/power/network hotspots with a **constraint solver**



Machines join the entitlement when job D and E start and return to the free pool when E stops (Source: Fig. 7)

TaskControl

```
service TaskController {
  TaskControlResponse process(TaskControlRequest request);
}
struct TaskControlRequest {
  string jobHandle;
  list<> request; // Pending task operations to be approved.
  list<> completed; // Completed task operations.
  list<> advanceNotices; // Upcoming planned maintenance.
  list<> allUnhealthyTasks; // Tasks unhealthy due to any reason.
  int sequenceNumber; // Increase after each call.
}
struct TaskControlResponse {
  list<> ack; // Approved task operations.
}
```



(b) Calling sequence of the TaskControl API when handling a job update. The job has two tasks: `t0` and `t1`. At time `S0`, the user initiates a job update. At time `S1`, the scheduler requests the approval of updates on tasks `t0` and `t1`, with `request=[t0,t1]`. The application's TaskController can selectively approve updates for any subset of tasks in any order. It approves the update on task `t1` by replying `ack=[t1]`, but delays the update on task `t0` to keep one task available. At time `S2`, the scheduler completes the update on task `t1` with `completed=[t1]`, and requests an update on the remaining task `t0`. This time, the TaskController approves the request.

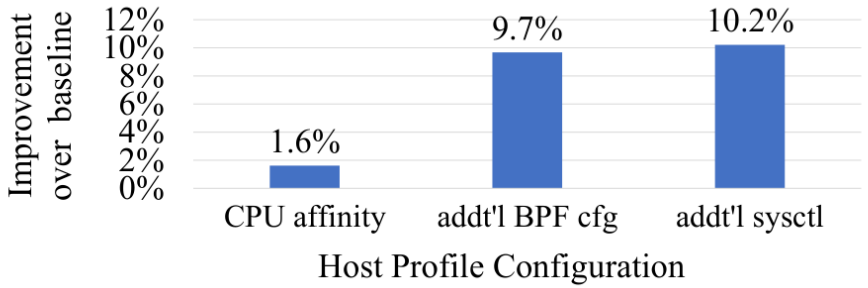
(Source: Tang et al., OSDI 2020, Fig. 8)

- Allow applications to collaborate with Twine when deciding which operations to proceed and which to postpone
- The scheduler sends **pending task operations**; the application's TaskController **acks any subset, in any order**. Right: it approves `t1` first and `t0` only after `t1` is healthy again, so one task is always serving

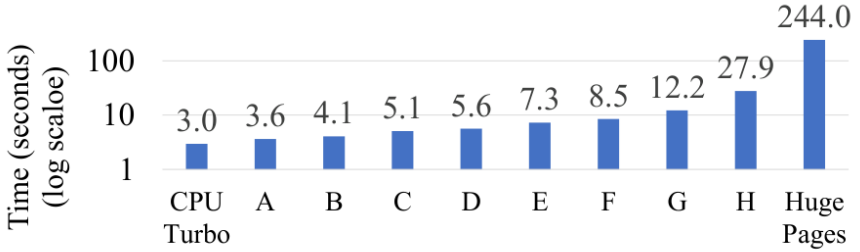
Host Profiles

- Some applications have special needs that would hurt most other services:
  - E.g. The web tier gains **11%** from OS settings (e.g., hugepages)
- A **host profile** = a set of hardware/OS settings (kernel, sysctls, cgroupv2, filesystem, NIC, CPU Turbo, HW prefetch); **one profile per entitlement**, applied by **Sidekick** whenever a machine changes entitlement
- A machine switches profile about **once every two days** → negligible overhead, except HugePages (**244 s** P90)
- **17 host profiles** and **16 TaskControllers** serve thousands of services

Memcache throughput (baseline 930K lookups/s)



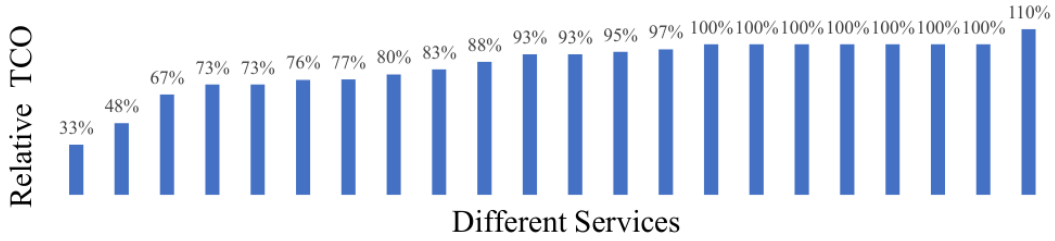
P90 time to switch a host profile



(Source: Tang et al., OSDI 2020, Figs. 16–17)

Small Machines + Autoscaling

- **Motivation**: maximize **performance per watt**
- **Big**: dual CPU, 256 GB RAM, dedicated NIC → **Small**: one 18-core CPU, 64 GB RAM, four per sled sharing a NIC.
- Same rack power budget: **92 small vs. 30 big → 57% more compute**
- Large services fill a small machine **without stacking**; **autoscaling** frees under-used machines



Relative TCO of 22 services on small vs. big machines; lower is better

(Source: Fig. 19)

Fitting into 64 GB took architecture changes: sharding (Shard Manager), moving in-memory data to external DBs, tiered memory over local flash (e.g., TAO via CacheLib)

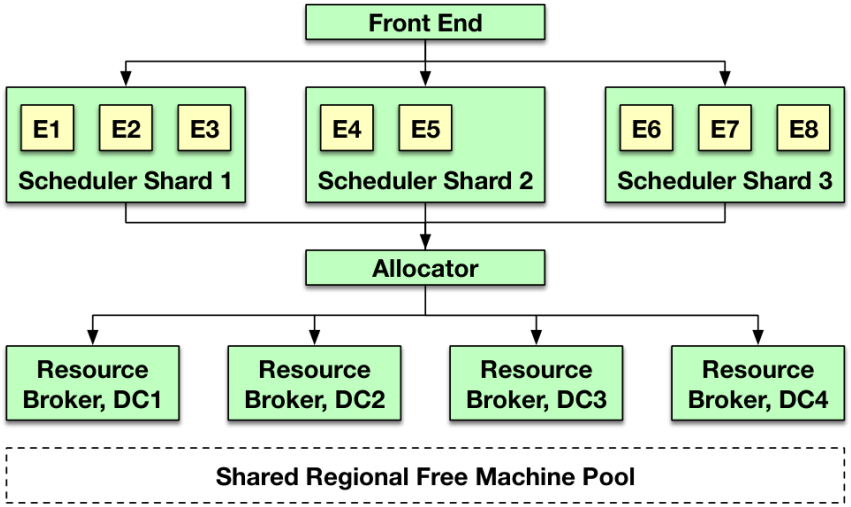
Scaling to One Million Machines

Principle 1: sharding

- Shard the **scheduler by entitlements**; entitlements can migrate between shards without restarting tasks (a job must fit in one shard — the largest has ≈15K tasks, so no limit in practice)
- Largest shard ≈ **170K machines** (≤40 cores, 80 GB); plan: **50K per shard → 20 shards for 1M machines** (a region's 150 MW)
- Allocator sharded 1:1 with the scheduler; ≈**1,000 job allocations/s** at peak

Principle 2: separation of concerns

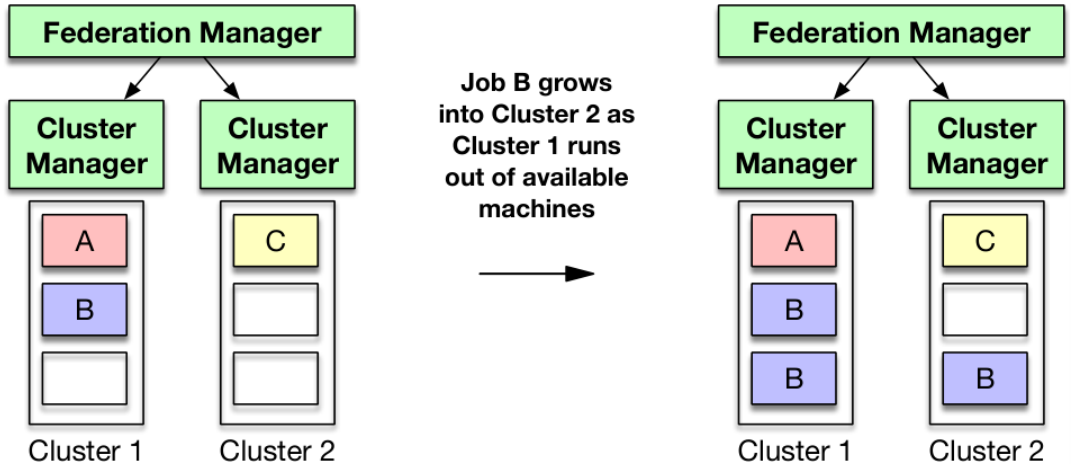
- Unlike Kubernetes' single API server + store (~5K machines), **every component is sharded**, each with its **own persistent store**
- **Allocation vs. optimization**: the allocator decides fast; ReBalancer optimizes off the critical path
- **Application-level schedulers** (batch, ML, stream, stateful) run on top; Twine never sees short batch jobs



(Source: Tang et al., OSDI 2020, Fig. 10)

Sharding vs. Federation

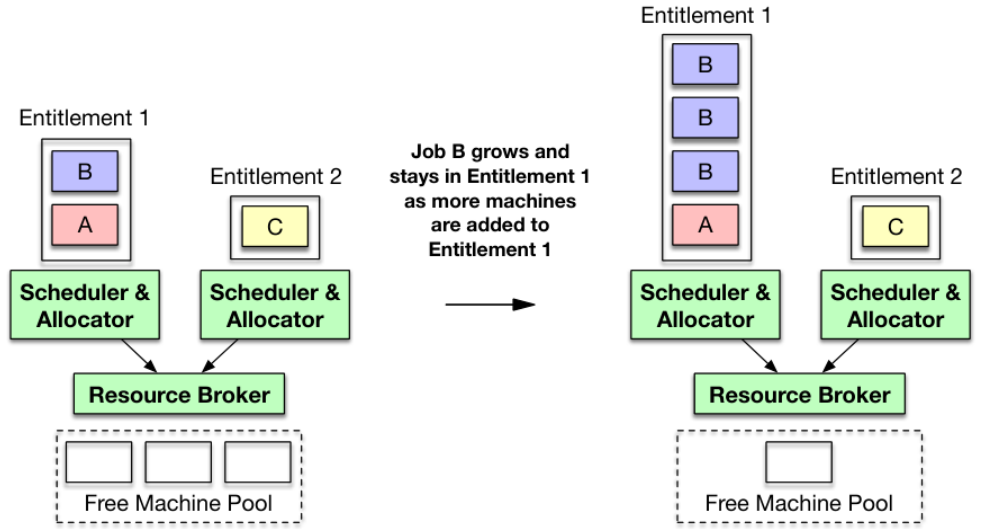
Federation (e.g., Kubernetes Federation)



Static clusters; a growing job is **split** across clusters by an extra manager layer

(Source: Tang et al., OSDI 2020, Fig. 13)

Twine's sharding



Dynamic entitlements; the job **stays put** and machines move to it from the free pool

Keeping a Regional Control Plane Reliable

New risks: a control-plane failure can affect a whole region, and a partitioned DC may be unreachable from its scheduler.

Design principles

- Every component is **sharded** (20 shards → 7.5 MW each, no bigger than a traditional cluster)
- Every component is **replicated** across DCs with leader election
- **Tasks keep running** even if all of Twine fails
- **Rate-limit destructive operations**
- Redundant regional network fabric

Operational principles

- **Twine manages itself** and its dependencies (ZooKeeper, Delos, Configerator) as Twine jobs
- **Gradual but frequent releases** (weekly or more), region by region, shard by shard
- **Recurring large-scale failure tests** in production